DCUtR 구현 가이드

개요

이 가이드는 Rust libp2p에서 DCUtR (Direct Connection Upgrade through Relay) 프로토콜을 구현하고 사용하는 방법을 상세히 설명합니다.

1. 기본 설정

1.1 의존성 추가

```
[dependencies]

libp2p = { version = "0.53", features = [

"dcutr",

"relay",

"identify",

"ping",

"tcp",

"noise",

"yamux",

"tokio"

]}

tokio = { version = "1.0", features = ["full"] }
```

```
tracing = "0.1"

tracing-subscriber = "0.3"
```

1.2 네트워크 동작 정의

```
use libp2p::{

dcutr, identify, ping, relay,

swarm::NetworkBehaviour,

};


#[derive(NetworkBehaviour)]

#[behaviour(prelude = "libp2p_swarm::derive_prelude")]

struct MyBehaviour {

relay_client: relay::client::Behaviour,

dcutr: dcutr::Behaviour,

identify: identify::Behaviour,

ping: ping::Behaviour,

}
```

2. Swarm 설정

2.1 기본 Swarm 구성

```
use libp2p::{
```

```
identity, noise, tcp, yamux,

SwarmBuilder,

};



async  fn create_swarm() -> Result<Swarm<MyBehaviour>, Box<dyn std::error::Error

let local_key = identity::Keypair::generate_ed25519();

let local_peer_id = local_key.public().to_peer_id();



let swarm = SwarmBuilder::with_existing_identity(local_key)

.with_tokio()

.with_tcp(

tcp::Config::default().nodelay(true),

noise::Config::new,

yamux::Config::default,

)?

.with_quic()

.with_dns()?

.with_relay_client(noise::Config::new, yamux::Config::default)?

.with_behaviour(|keypair, relay_behaviour| MyBehaviour {

relay_client: relay_behaviour,

dcutr: dcutr::Behaviour::new(keypair.public().to_peer_id()),

identify: identify::Behaviour::new(identify::Config::new(

"/my-app/1.0.0".to_string(),
```

```
keypair.public(),
)),
ping: ping::Behaviour::new(ping::Config::new()),
})?
.build();


Ok(swarm)
}
```

2.2 리스닝 주소 설정

```
// TCP와 QUIC 모두에서 리스닝
swarm.listen_on("/ip4/0.0.0.0/tcp/0".parse()?)?;
swarm.listen_on("/ip4/0.0.0.0/udp/0/quic-v1".parse()?)?;
```

3. 릴레이 서버 연결

3.1 릴레이 서버 연결 설정

```
async fn connect_to_relay(
swarm: &mut Swarm<MyBehaviour>,
relay_address: Multiaddr,
) -> Result<(), Box<dyn std::error::Error>> {
```

```
println!("릴레이 서버에 연결 중: {}", relay_address);

// 릴레이 서버에 연결

swarm.dial(relay_address.clone())?;

// 연결 완료 대기

loop {

match swarm.select_next_some().await {

SwarmEvent::ConnectionEstablished { peer_id, .. } => {

println!("릴레이 서버 연결 완료: {}", peer_id);

break;

}

SwarmEvent::Behaviour(MyBehaviourEvent::Identify(

identify::Event::Received { info, .. }

)) => {

println!("관찰된 주소: {}", info.observed_addr);

}

_ => {}

}

}

Ok(())

}
```

4. 리스너 모드 구현

4.1 릴레이 예약 요청

```
async  fn setup_relay_listener(
swarm: &mut Swarm<MyBehaviour>,
relay_address: Multiaddr,
) -> Result<(), Box<dyn std::error::Error>> {
// 릴레이를 통한 리스닝 시작
let relay_listen_addr = relay_address.with(Protocol::P2pCircuit);
swarm.listen_on(relay_listen_addr.clone())?;
println!("릴레이 리스닝 주소: {}", relay_listen_addr);
// 예약 승인 대기
loop {
match swarm.select_next_some().await {
SwarmEvent::NewListenAddr { address, .. } => {
println!("새 리스닝 주소: {}", address);
}
SwarmEvent::Behaviour(MyBehaviourEvent::RelayClient(
relay::client::Event::ReservationReqAccepted { .. }
)) => {
println!("릴레이 예약 승인됨");
break;
}
SwarmEvent::Behaviour(MyBehaviourEvent::RelayClient(
```

```
relay::client::Event::ReservationReqFailed { error, .. }
)) => {
eprintln!("릴레이 예약 실패: {:?}", error);
return Err(error.into());
}
_ => {}
}
}
Ok(())
}
```

4.2 DCUtR 이벤트 처리

```
async  fn handle_dcutr_events(
swarm: &mut Swarm<MyBehaviour>,
) -> Result<(), Box<dyn std::error::Error>> {
loop {
match swarm.select_next_some().await {
SwarmEvent::Behaviour(MyBehaviourEvent::Dcutr(dcutr::Event {
remote_peer_id,
result,
})) => {
match result {
```

```
Ok(connection_id) => {
println!(
"DCUtR 홀 펀칭 성공! 피어: {}, 연결 ID: {:?}",
remote_peer_id, connection_id
);
}
Err(error) => {
eprintln!(
"DCUtR 홀 펀칭 실패! 피어: {}, 오류: {:?}",
remote_peer_id, error
);
}
}
}
SwarmEvent::ConnectionEstablished {
peer_id,
endpoint,
connection_id,
..
} => {
println!(
"연결 설정됨 - 피어: {}, 엔드포인트: {:?}, 연결 ID: {:?}",
peer_id, endpoint, connection_id
```

```
);

// 직접 연결인지 확인

if !endpoint.get_remote_address().iter()

.any(|p| p == Protocol::P2pCircuit) {

println!("직접 P2P 연결 설정됨!");

}

}

_ => {}

}

}

}
```

5. 다이얼러 모드 구현

5.1 릴레이를 통한 연결

```
async  fn dial_through_relay(

swarm: &mut Swarm<MyBehaviour>,

relay_address: Multiaddr,

target_peer_id: PeerId,

) -> Result<(), Box<dyn std::error::Error>> {

// 릴레이를 통한 대상 피어 연결

let relay_dial_addr = relay_address
```

```
.with(Protocol::P2pCircuit)

.with(Protocol::P2p(target_peer_id));

println!("릴레이를 통한 다이얼: {}", relay_dial_addr);

swarm.dial(relay_dial_addr)?;

// 연결 및 DCUtR 이벤트 처리

loop {

match swarm.select_next_some().await {

SwarmEvent::ConnectionEstablished { peer_id, .. }

if peer_id == target_peer_id => {

println!("릴레이 연결 설정됨, DCUtR 홀 펀칭 시작 예상");

}

SwarmEvent::Behaviour(MyBehaviourEvent::Dcutr(dcutr::Event {

remote_peer_id,

result,

})) if remote_peer_id == target_peer_id => {

match result {

Ok(connection_id) => {

println!("홀 펀칭 성공! 연결 ID: {:?}", connection_id);

return Ok(());

}

Err(error) => {

eprintln!("홀 펀칭 실패: {:?}", error);

return Err(error.into());
```

```
}
}
}
_ => {}
}
}
}
```

6. 고급 설정

6.1 DCUtR 설정 커스터마이징

```
// DCUtR 동작은 현재 설정 옵션이 제한적이지만,
// 최대 시도 횟수는 내부적으로 3회로 고정됨
let dcutr_behaviour = dcutr::Behaviour::new(local_peer_id);
```

6.2 주소 후보 관리

```
// 외부 주소 후보 추가 (identify 프로토콜을 통해 자동으로 수집됨)
SwarmEvent::NewExternalAddrCandidate { addr } => {
println!("새 외부 주소 후보: {}", addr);
// DCUtR은 자동으로 이 주소들을 홀 펀칭에 사용
}
```

6.3 연결 상태 모니터링

```
#[derive(Debug)]
struct ConnectionMonitor {
direct_connections: HashMap<PeerId, Vec<ConnectionId>>,
relayed_connections: HashMap<PeerId, Vec<ConnectionId>>,
}

impl ConnectionMonitor {
fn new() -> Self {
Self {
direct_connections: HashMap::new(),
relayed_connections: HashMap::new(),
}
}
fn add_connection(&mut  self, peer_id: PeerId, connection_id: ConnectionId, endp
let is_relayed = endpoint.get_remote_address().iter()
.any(|p| p == Protocol::P2pCircuit);
if is_relayed {
self.relayed_connections.entry(peer_id).or_default().push(connection_id);
} else {
self.direct_connections.entry(peer_id).or_default().push(connection_id);
}
```

```
}

fn has_direct_connection(&self, peer_id: &PeerId) -> bool {

self.direct_connections.get(peer_id)

.map(|conns| !conns.is_empty())

.unwrap_or(false)

}

}
```

7. 오류 처리

7.1 일반적인 오류 상황

```
async fn handle_dcutr_errors(error: &dcutr::Error) {

match error {

dcutr::Error { .. } => {

eprintln!("DCUtR 오류: {}", error);

// 오류 유형에 따른 처리

if error.to_string().contains("AttemptsExceeded") {

println!("최대 시도 횟수 초과 - 나중에 다시 시도");

} else if error.to_string().contains("InboundError") {

println!("인바운드 스트림 오류 - 네트워크 연결 확인");

} else if error.to_string().contains("OutboundError") {

println!("아웃바운드 스트림 오류 - 대상 피어 상태 확인");
```

```
}
}
}
}
```

7.2 재시도 로직

```
async fn retry_hole_punching(
swarm: &mut Swarm<MyBehaviour>,
target_peer: PeerId,
max_retries: u32,
) -> Result<(), Box<dyn std::error::Error>> {
for attempt in 1..=max_retries {
println!("홀 펀칭 시도 #{}", attempt);
match attempt_hole_punch(swarm, target_peer).await {
Ok(_) => {
println!("홀 펀칭 성공!");
return Ok(());
}
Err(e) => {
eprintln!("시도 #{} 실패: {}", attempt, e);
if attempt < max_retries {
tokio::time::sleep(Duration::from_secs(5)).await;
```

```
}
}
}
}
Err("모든 홀 펀칭 시도 실패".into())
}

async  fn attempt_hole_punch(
swarm: &mut Swarm<MyBehaviour>,
target_peer: PeerId,
) -> Result<(), Box<dyn std::error::Error>> {
// 홀 펀칭 시도 로직
// (실제 구현에서는 DCUtR이 자동으로 처리)
Ok(())
}
```

8. 성능 최적화

8.1 주소 후보 우선순위

```
// identify 프로토콜을 통해 수집된 주소들은 자동으로 우선순위가 매겨짐
// 더 자주 관찰되는 주소가 높은 우선순위를 가짐
```

```
SwarmEvent::Behaviour(MyBehaviourEvent::Identify(
identify::Event::Received { info, .. }
)) => {
// 관찰된 주소는 자동으로 DCUtR 주소 후보에 추가됨
println!("관찰된 주소 (홀 펀칭에 사용됨): {}", info.observed_addr);
}
```

8.2 연결 관리

```
// 불필요한 릴레이 연결 정리
SwarmEvent::Behaviour(MyBehaviourEvent::Dcutr(dcutr::Event {
result: Ok(direct_connection_id),
remote_peer_id,
})) => {
println!("직접 연결 성공, 릴레이 연결 정리 고려");
// 필요에 따라 릴레이 연결을 유지하거나 종료
// (libp2p는 자동으로 중복 연결을 관리함)
}
```

9. 완전한 예제

9.1 리스너 애플리케이션

```
use libp2p::{
```

```
core::multiaddr::Protocol,

dcutr, identify, ping, relay,

swarm::{NetworkBehaviour, SwarmEvent},

Multiaddr, SwarmBuilder,

};


#[derive(NetworkBehaviour)]

#[behaviour(prelude = "libp2p_swarm::derive_prelude")]

struct ListenerBehaviour {

relay_client: relay::client::Behaviour,

dcutr: dcutr::Behaviour,

identify: identify::Behaviour,

ping: ping::Behaviour,

}


#[tokio::main]

async  fn main() -> Result<(), Box<dyn std::error::Error>> {

tracing_subscriber::fmt::init();


let  mut swarm = SwarmBuilder::with_new_identity()

.with_tokio()

.with_tcp(

libp2p::tcp::Config::default(),
```

```
libp2p::noise::Config::new,
libp2p::yamux::Config::default,
)?
.with_relay_client(
libp2p::noise::Config::new,
libp2p::yamux::Config::default,
)?
.with_behaviour(|keypair, relay_client| ListenerBehaviour {
relay_client,
dcutr: dcutr::Behaviour::new(keypair.public().to_peer_id()),
identify: identify::Behaviour::new(identify::Config::new(
"/dcutr-listener/1.0.0".to_string(),
keypair.public(),
)),
ping: ping::Behaviour::new(ping::Config::new()),
})?
.build();


let local_peer_id = *swarm.local_peer_id();
println!("로컬 피어 ID: {}", local_peer_id);


// 로컬 주소에서 리스닝
swarm.listen_on("/ip4/0.0.0.0/tcp/0".parse()?)?;
```

```
// 릴레이 서버 주소 (실제 환경에서는 실제 릴레이 서버 주소 사용)
let relay_address: Multiaddr = "/ip4/127.0.0.1/tcp/4001/p2p/12D3KooWDpJ7As7BWAwR
.parse()?;

// 릴레이 서버 연결
swarm.dial(relay_address.clone())?;

// 릴레이를 통한 리스닝 설정
swarm.listen_on(relay_address.with(Protocol::P2pCircuit))?;

loop {
match swarm.select_next_some().await {
SwarmEvent::NewListenAddr { address, .. } => {
println!("리스닝 주소: {}", address);
}
SwarmEvent::Behaviour(ListenerBehaviourEvent::RelayClient(
relay::client::Event::ReservationReqAccepted { .. }
)) => {
println!("릴레이 예약 승인됨 - 연결 대기 중");
}
SwarmEvent::Behaviour(ListenerBehaviourEvent::Dcutr(dcutr::Event {
remote_peer_id,
```

```
result,
})) => {
match result {
Ok(connection_id) => {
println!("홀 펀칭 성공! 피어: {}, 연결: {:?}",
remote_peer_id, connection_id);
}
Err(error) => {
eprintln!("홀 펀칭 실패! 피어: {}, 오류: {}",
remote_peer_id, error);
}
}
}
SwarmEvent::ConnectionEstablished { peer_id, endpoint, .. } => {
let is_relayed = endpoint.get_remote_address().iter()
.any(|p| p == Protocol::P2pCircuit);
if is_relayed {
println!("릴레이 연결 설정됨: {}", peer_id);
} else {
println!("직접 연결 설정됨: {}", peer_id);
}
}
_ => {}
```

```
}

}

}
```

9.2 다이얼러 애플리케이션

```
// 다이얼러 구현은 리스너와 유사하지만,

// 릴레이를 통해 특정 피어에 연결을 시도함


#[tokio::main]

async  fn main() -> Result<(), Box<dyn std::error::Error>> {

// ... (swarm 설정은 리스너와 동일)

let target_peer_id: PeerId = "12D3KooWExample...".parse()?;

let relay_address: Multiaddr = "/ip4/127.0.0.1/tcp/4001/p2p/12D3KooWRelay...".pa

// 릴레이를 통한 대상 피어 연결

let dial_addr = relay_address

.with(Protocol::P2pCircuit)

.with(Protocol::P2p(target_peer_id));

swarm.dial(dial_addr)?;

// 이벤트 처리 루프 (리스너와 동일)

// ...
```

```
}
```

10. 트러블슈팅

10.1 일반적인 문제들

1. **홀 펀칭 실패**

- NAT 타입 확인 (Symmetric NAT는 홀 펀칭이 어려움)
- 방화벽 설정 확인
- 릴레이 서버 연결 상태 확인

2. **릴레이 예약 실패**

- 릴레이 서버 용량 확인
- 네트워크 연결 상태 확인
- 릴레이 서버 주소 정확성 확인

3. **주소 발견 문제**

- identify 프로토콜 동작 확인
- 외부 주소 후보 수집 상태 확인

10.2 디버깅 팁

```
// 상세한 로깅 활성화
tracing_subscriber::fmt()
.with_env_filter("libp2p_dcutr=debug,libp2p_relay=debug")
.init();
```

```
// 연결 상태 모니터링

SwarmEvent::ConnectionEstablished { peer_id, endpoint, .. } => {

println!("연결 설정: {} via {}", peer_id, endpoint.get_remote_address());

}



SwarmEvent::ConnectionClosed { peer_id, cause, .. } => {

println!("연결 종료: {} (원인: {:?})", peer_id, cause);

}
```

이 가이드를 통해 DCUtR 프로토콜을 효과적으로 구현하고 사용할 수 있습니다. 실제 프로덕션 환경에서는 네트워크 조건과 NAT 타입에 따라 성공률이 달라질 수 있으므로, 적절한 폴백 메커니즘을 구현하는 것이 중요합니다.